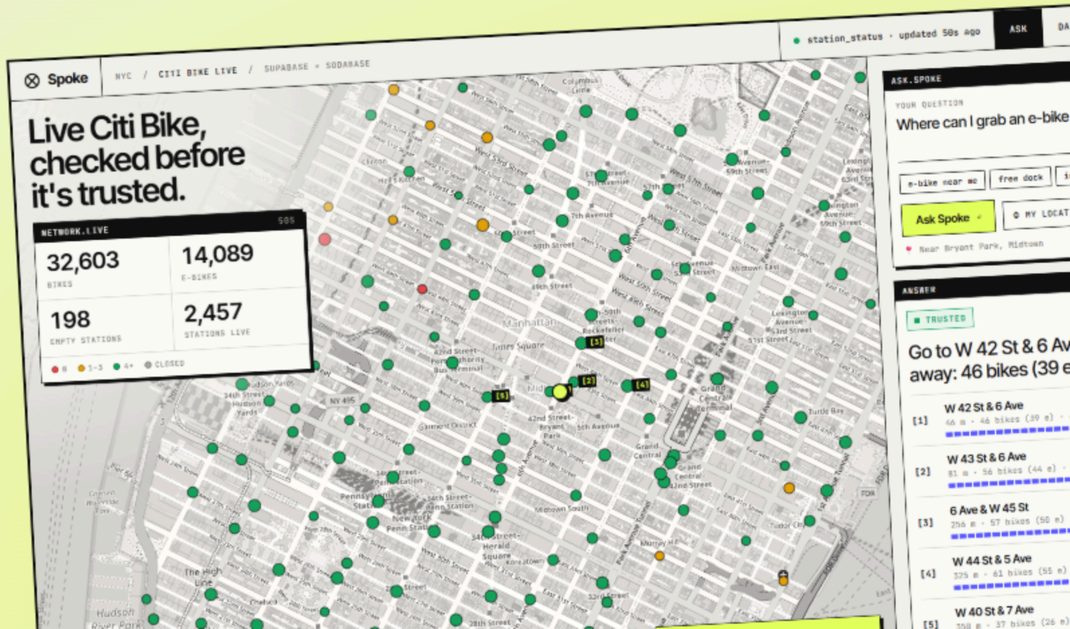
SUPABASE + SODABASE NYC HACKATHON

Spoke

Live Citi Bike, checked before it's trusted.

Pulled into Supabase every minute by pg_cron and an edge function. Checked against a data contract. Watched by Sodabase. And an AI answer that refuses to guess when the data can't be trusted.



Paras Adlakha · Harshita Adlakha · 4 November 2026 · New York

THE PROBLEM

The app said three bikes. You walked ten minutes. The dock was empty.

Citi Bike's public feed powers hundreds of apps and maps: ~2,500 stations, a record 44.3 million trips in 2024. It's good data, but not clean data. One pull on 4 November:

48

stations last reported in 1970

`last_reported: 86400` is a placeholder for 2 Jan 1970. Pass it through and "updated 56 years ago" breaks every freshness check.

9

docks over capacity

"18 bikes + docks at a 4-dock station." Counts that can't physically be true.

5

"installed", silent for a day

Marked open, but nothing reported in over 24 hours. Is the dock empty, or the data dead?

Source: Citi Bike GBFS 2.3, profiled 4 Nov 2026 19:05 UTC

WHAT SPOKE DOES

Ask. Spoke checks the data first, then answers.

■ TRUSTED

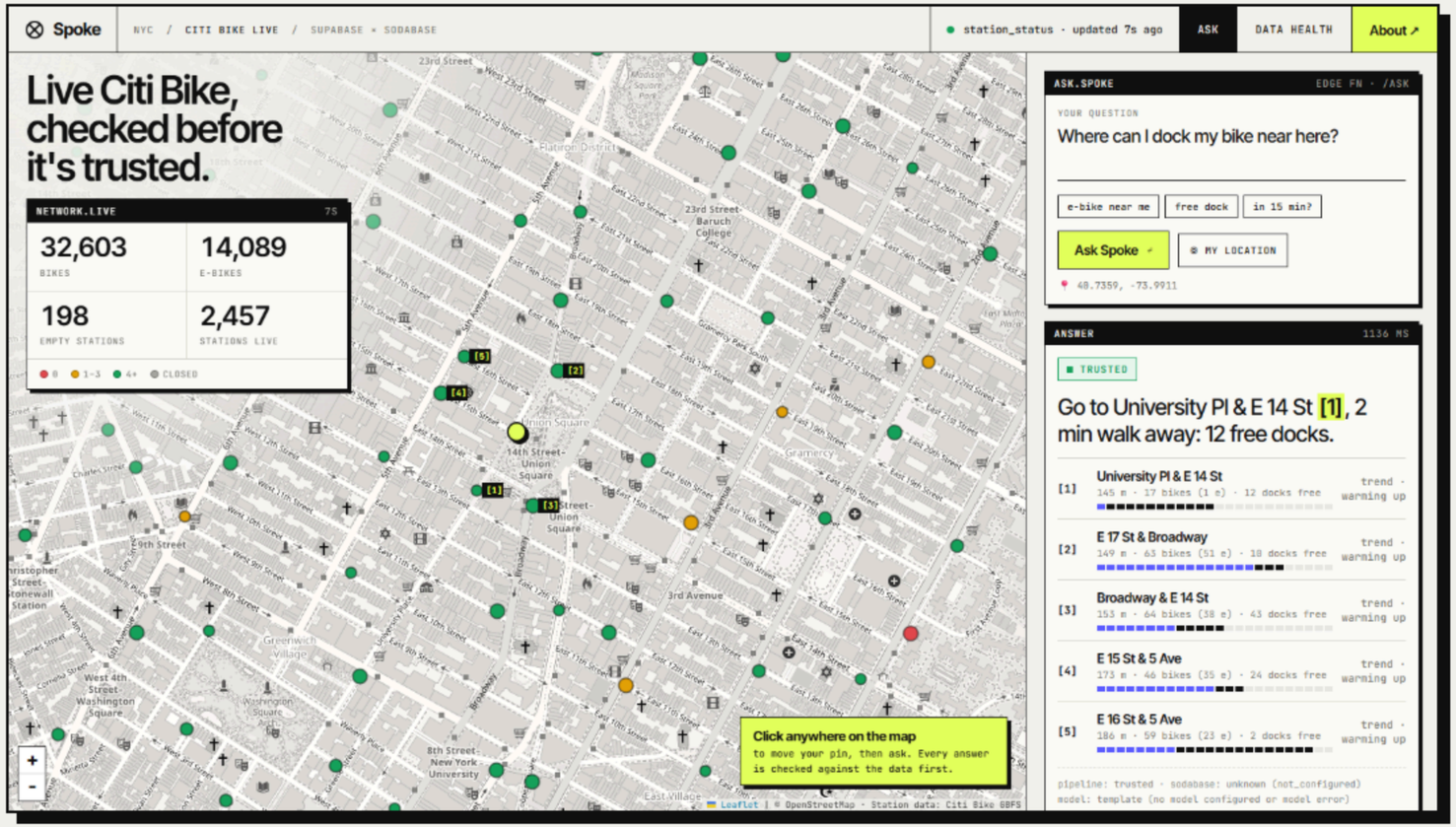
Nearest 5 stations, live counts and 60-minute trend. Each one cited as [n].

■ CAVEATED

Answers, and says why the numbers may be off.

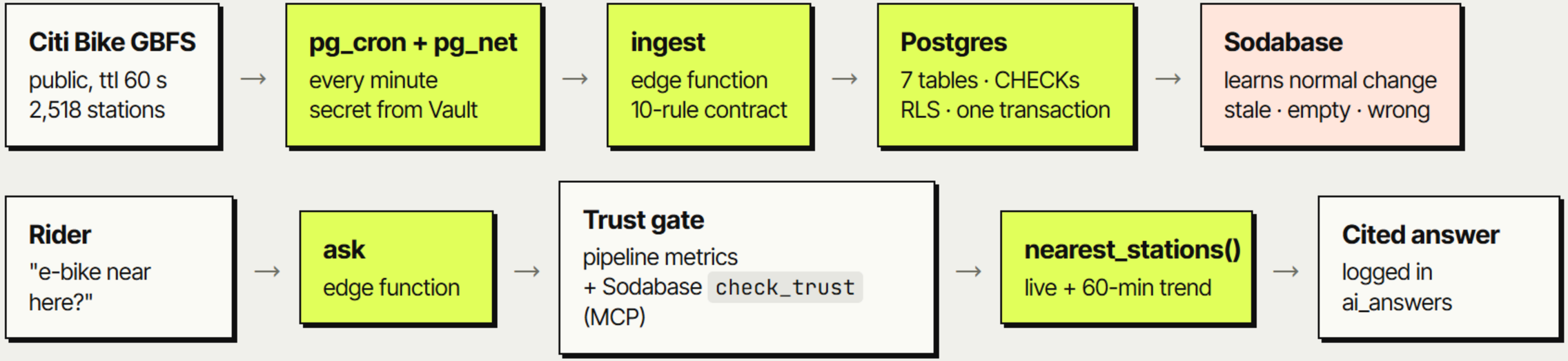
■ UNTRUSTED

"I won't guess. Live data is 12 min old." No model call at all.



HOW IT'S BUILT · ALL SUPABASE

A database with two small functions on top.



PG_CRON JOB	SCHEDULE	DOES
`spoke-ingest-status`	`* * * * *`	live bikes and docks → `station_status`, `system_pulse`, `ingest_runs`
`spoke-ingest-stations`	`7 * * * *`	names, location, capacity → `stations`
`spoke-snapshot`	`*/10 * * * *`	pure SQL history → `status_snapshots` (trends)

THE DATA CONTRACT

Bad rows never reach the live tables. They're quarantined, with a reason.

RULE	LIVE, 4 NOV
`bogus_timestamp` · before Citi Bike existed	**48**
`over_capacity` · bikes + docks > capacity + 2	**9–15**
`future_timestamp` · `unknown_station` · `duplicate_station`	0
`invalid_count` · `ebikes_exceed_bikes` · `invalid_flag`	0
`outside_service_area` · `missing_name` · `invalid_capacity`	0

Postgres `CHECK` constraints repeat the critical rules. Every run, even a failed one, writes an `ingest_runs` row, so a silent pipeline shows up as stale data.

`SELECT * FROM rejected_records`

`bogus_timestamp` **× 48 stations**
last_reported 86400 is before Citi Bike existed

`over_capacity`
31 bikes + docks at a 22-dock station

`over_capacity`
29 bikes + docks at a 24-dock station

`over_capacity`
42 bikes + docks at a 35-dock station

One row per station and reason, with `times_seen` counting up each minute: no table bloat.

BUILT FOR SODABASE

Designed so "normal" is easy to learn, and a problem is easy to see.

■ STALE

A heartbeat every minute

`ingest_runs` and `system_pulse` insert one row a minute; ~1,800 stations re-report in `station_status`. A stopped cron shows within minutes.

■ EMPTY

Nothing is ever deleted

History is insert-only; live tables are upserted in place. No truncation, so a row-count drop is always a real problem.

■ WRONG

Contract + constraints

Schema only changes through migrations. Bad values are blocked twice: in TypeScript, then by Postgres.

Sodabase closes the loop. Every answer calls Sodabase's MCP `check_trust` on `public.station_status`. If Sodabase says untrusted, Spoke won't answer, however healthy its own pipeline looks.

WHAT'S NEXT

From "is the data fresh?" to "will there be a bike?"

Predict

Hour-of-week availability curves per station from `status_snapshots`: "this dock empties by 8:40 on weekdays."

Alert the source

Send `rejected_records` to the feed operator: 48 docks reporting 1970 is fixable upstream.

Every GBFS city

The same contract works for Divvy, Bay Wheels and Capital Bikeshare. GBFS is a standard.

Push, not pull

"Your usual dock is draining fast" before you leave the office.